CÓDIGOCLARO · RUTA PYTHON

Tema 09 · Módulos y paquetes

Cuaderno de estudio para DAM/DAW · 10 lecciones

Qué vas a aprender

import, __name__, paquetes y organización.

Este cuaderno trabaja módulos y paquetes en pasos cortos. La meta no es repetir palabras de memoria: es reconocer el problema, elegir una herramienta adecuada, explicar por qué funciona y comprobarla con datos distintos. Reserva una sesión para leer el modelo, otra para resolver el ejercicio y una tercera para intentar el laboratorio sin mirar la solución.

Resultados de aprendizaje

- Explicar con tus palabras qué problema resuelve módulos y paquetes y en qué parte de una aplicación DAM/DAW aparece.
- Leer el ejemplo, predecir el resultado y justificar cada decisión relevante.
- Resolver una práctica, comprobar casos normales y límite, y depurar un fallo con un método reproducible.

Cómo estudiar este tema

Ten a mano un editor compatible con la tecnología y copia los ejemplos en un archivo nuevo. Antes de ejecutar, escribe tu predicción. Si no coincide con la salida, no borres el intento: anota qué supuesto era incorrecto y prueba una variación mínima. Al terminar, explica el concepto a otra persona usando un ejemplo propio.

Punto de control inicial

Antes de avanzar, escribe dos ideas que ya conoces sobre este tema y una pregunta que todavía no sabes responder. Vuelve a ellas al final de la lección 10. Si cambias de respuesta, explica qué ejemplo o prueba hizo que cambiaras de opinión.

Lección 1 · El modelo mental

Idea central

Un módulo es normalmente un archivo .py que contiene definiciones importables. import enlaza el módulo o nombres elegidos al espacio actual. Cuando Python ejecuta un archivo directamente, su variable __name__ vale __main__; al importarlo, __name__ toma el nombre del módulo. El bloque de guarda permite ofrecer una acción de demostración sin ejecutarla en cada importación. Un paquete agrupa módulos y su organización debe mantener nombres claros.

Para razonar sobre módulos y paquetes, separa cuatro preguntas: qué datos existen al empezar, qué regla se aplica, qué cambia durante el proceso y qué resultado debe observar quien usa el programa. Esta separación ayuda a no confundir la sintaxis con la finalidad. Una línea puede compilar y aun así resolver mal el problema si el dato representa otra cosa o falta un caso.

En Python, ten presente este marco

Python asocia los tipos a los objetos y permite que una variable nombre valores de tipos distintos. int, float, str, bool y None aparecen desde el inicio; list, dict y set son mutables, mientras que tuple y str no se modifican en el sitio. La indentación delimita bloques. input() devuelve texto, por lo que convertir y validar son pasos diferentes.

Comprueba tu comprensión

- ¿Qué parte de la explicación define la regla principal de módulos y paquetes?
- ¿Qué dato puede cambiar y qué dato debería permanecer estable?
- ¿Qué observación concreta demostraría que tu programa cumple el objetivo?

Lección 2 · Vocabulario y reglas

Reglas de Python que afectan a este tema

Python asocia los tipos a los objetos y permite que una variable nombre valores de tipos distintos. int, float, str, bool y None aparecen desde el inicio; list, dict y set son mutables, mientras que tuple y str no se modifican en el sitio. La indentación delimita bloques. input() devuelve texto, por lo que convertir y validar son pasos diferentes.

Ideas específicas de módulos y paquetes

Un módulo establece un límite de organización: qué nombres expone, qué depende de otros y qué detalle permanece interno. Mantén dependencias dirigidas de forma comprensible y evita ciclos innecesarios. Separa configuración del código, registra cómo se instala el proyecto y comprueba que una importación funciona desde el punto de entrada real, no solo desde el directorio del editor.

Glosario de trabajo

Término	Significado en este recorrido
Módulo	Unidad de código que puede exponer nombres y depender de otros.
Importación	Acceso a nombres definidos en otro módulo.
Dependencia	Componente externo o interno que otro necesita para funcionar.
Punto de entrada	Lugar desde el que comienza el programa o una herramienta.

No memorices la tabla como definiciones aisladas. Relaciona cada término con una línea del ejemplo o una decisión del ejercicio. Si una palabra aparece con otro significado en una API o en otra versión del lenguaje, anótalo junto a la fuente oficial antes de aplicarla al proyecto.

Lección 3 · Lectura guiada del ejemplo

Primero identifica la entrada, luego sigue el valor por el programa. No supongas que una asignación transforma el dato: a veces solo enlaza un nombre con el mismo objeto. Observa dónde empieza cada bloque, qué condición activa una rama y qué instrucción produce la salida. Si el ejemplo tiene varias líneas, ejecútalo por partes y añade temporalmente observaciones seguras para ver los valores intermedios.

Código · python

```
def doble(numero):
    return numero * 2

if __name__ == "__main__":
    print(doble(4))
```

Contexto: Guarda el código como operaciones.py. Al importarlo, doble queda disponible y el bloque principal no imprime nada.

Salida esperada

```
8
```

Registro de la traza

Momento	Qué anotar
Antes de ejecutar	Valores iniciales, sus tipos y qué operación esperas que ocurra.
Después de cada paso	Nombre afectado, valor nuevo y regla que produjo el cambio.
Al encontrar una decisión	Condición evaluada y camino que se ejecutó.
Al terminar	Salida real, diferencia respecto a tu predicción y causa.
Con una entrada distinta	Qué resultado cambia y qué parte del contrato permanece igual.

Traza en papel

- Antes de la primera instrucción: apunta los valores iniciales y sus tipos.
- Después de cada operación: anota solo los nombres que hayan cambiado y la razón.
- Al final: compara tu predicción con «8» y explica la diferencia si la hay.

Cambia un dato de entrada y vuelve a hacer la traza. Una buena explicación debe anticipar el nuevo resultado, no limitarse a repetir la salida original.

Lección 4 · Decidir cuándo usarlo

Un módulo establece un límite de organización: qué nombres expone, qué depende de otros y qué detalle permanece interno. Mantén dependencias dirigidas de forma comprensible y evita ciclos innecesarios. Separa configuración del código, registra cómo se instala el proyecto y comprueba que una importación funciona desde el punto de entrada real, no solo desde el directorio del editor.

Caso aplicado: en un proyecto de DAM/DAW, una pantalla de matrículas necesita módulos y paquetes. Describe qué datos de la matrícula intervienen, qué regla del tema usarías y qué resultado verá el usuario. Mantén el cálculo o decisión separado de la lectura de interfaz cuando sea posible; así podrás comprobar la regla sin abrir la pantalla.

Compara antes de elegir

- ¿La solución más directa deja claro el propósito o requiere conocer demasiados detalles internos?
- ¿Qué coste aparece si crece el volumen de datos o se añade un nuevo caso?
- ¿Qué comprobación automatizada detectaría una modificación accidental?

Enfoques relacionados

- Importar el módulo y llamar operaciones.convertir.
- Importar solo convertir con from operaciones import convertir.

No existe una alternativa mejor en todos los contextos. Elige una y escribe su ventaja y su límite para el tamaño, la versión y los requisitos del ejercicio. Si dos enfoques cumplen exactamente el mismo contrato y ambos son legibles, pueden considerarse válidos.

Lección 5 · Errores y diagnóstico

Fallos que conviene reconocer

- Llamar al archivo como un módulo de la biblioteca estándar que intenta importar. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Ejecutar consultas, impresiones o cambios globales al importar un módulo. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Confundir la ruta del archivo actual con la carpeta desde la que se ejecuta Python. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.

Rutina de depuración

1. Describe el resultado esperado y el resultado real sin mezclar ambos. 2. Reproduce el fallo con una entrada fija. 3. Reduce el programa hasta dejar la primera operación que se comporta distinto. 4. Formula una hipótesis comprobable y cambia una sola cosa. 5. Añade una prueba que impida que el mismo fallo vuelva. Este orden también funciona cuando el error aparece en una API, una interfaz o un archivo.

Preguntas de revisión

- ¿El error está en los datos, en la regla o en cómo se presenta el resultado?
- ¿Qué caso límite no estaba cubierto por el ejemplo inicial?
- ¿La solución corrige la causa o solo oculta el síntoma?

Lección 6 · Práctica guiada

Situación de DAM/DAW

Un pequeño programa separa una operación reutilizable de la presentación del resultado para que pueda importarse en otro ejercicio.

Enunciado

Divide una operación de conversión en un módulo y un archivo principal que la importe. La importación no debe imprimir ni pedir datos.

Convierte el enunciado en requisitos verificables

- Escribe qué entradas acepta la solución y qué debe devolver o mostrar.
- Enumera las reglas de negocio y separa las que son obligatorias de las decisiones de presentación.
- Antes de programar, completa la tabla de pruebas y predice cada resultado.

Casos que debe superar

Entrada	Resultado esperado
convertir(0)	0
convertir(3)	6
convertir(-2)	-4

Qué demuestra cada caso

Clasifica cada fila de la tabla como caso habitual, límite o variación importante. Si el enunciado no define qué hacer con una entrada fuera de contrato, no inventes un resultado silencioso: anota la ambigüedad y propone una regla antes de programar. En la práctica profesional, acordar esa regla evita que dos implementaciones parezcan correctas y se comporten de forma incompatible.

Plan de solución

- Divide la tarea en pasos pequeños que puedan comprobarse por separado.
- Implementa primero el caso habitual y después añade los límites y errores previstos.
- Ejecuta cada caso tras un cambio; no esperes al final para descubrir qué parte falló.

Lección 7 · Solución explicada

Una solución no se justifica solo porque la salida coincida en un ejemplo. Debe respetar el contrato para todas las entradas admitidas. Lee el código siguiente y vincula cada operación con una frase del enunciado. Si no puedes señalar esa relación, revisa el diseño antes de continuar.

Código · python

```
# operaciones.py
def convertir(numero):
    return numero * 2

# principal.py
from operaciones import convertir

def main():
    print(convertir(3))

if __name__ == "__main__":
    main()
```

Criterio de aceptación: Importar operaciones o principal no imprime ni pide datos; ejecutar principal.py muestra 6.

Por qué se sigue este razonamiento

operaciones.py define convertir sin entrada ni salida. principal.py también se puede importar en silencio porque su llamada a main queda dentro de la guarda __main__; al ejecutarlo directamente, muestra 6. La función produce 0, 6 y -4 para los tres casos.

Comprobación de corrección

Comprueba cada caso de la tabla y explica qué parte de la solución lo cubre. Después revisa el caso mínimo, el máximo razonable y una entrada inválida si el contrato la permite. Comprueba también los tipos de entrada y salida: una coincidencia textual puede ocultar un valor mal tipado o una conversión con pérdida.

Paso	Justificación que debes poder dar
Contrato	Qué valores acepta el ejercicio y qué resultado promete.
Regla	Qué operación transforma la entrada en la salida.
Pruebas	Qué caso comprueba lo normal y cuál descubre un límite.

Alternativas válidas

- Importar el módulo y llamar operaciones.convertir.
- Importar solo convertir con from operaciones import convertir.

Al comparar soluciones, prioriza contrato correcto, legibilidad y pruebas. Una alternativa que cambia la representación interna puede ser válida; una que cambia silenciosamente el significado de la salida no lo es.

Lección 8 · Laboratorio de aplicación

Retoma el ejercicio de módulos y paquetes y amplíalo como una pequeña funcionalidad de una aplicación de gestión académica. Conserva el requisito original y añade una regla nueva que tenga sentido para los datos. Antes de modificar el código, escribe la regla en lenguaje natural, un ejemplo que la cumpla y otro que la incumpla.

Variación para transferir lo aprendido

Importa convertir desde principal.py y captura la salida estándar; determina si el requisito exige que también importar el archivo principal sea silencioso.

Desafíos progresivos

- Nivel 1: añade un caso límite que no aparecía en la tabla y registra el resultado esperado.
- Nivel 2: cambia la forma de recibir o recorrer los datos sin cambiar el contrato externo.
- Nivel 3: separa la regla principal de su entrada/salida y prueba ambas partes de forma independiente.

Criterios de entrega

- La solución conserva el resultado esperado del ejercicio original.
- El nuevo comportamiento queda cubierto por una prueba que falla antes del cambio.
- Los nombres permiten seguir el flujo sin comentarios que repitan la sintaxis.
- La entrada inválida o el error externo tiene una respuesta definida.

Prueba de regresión

Guarda los casos originales antes de añadir la regla nueva. Ejecuta la batería completa después de cada cambio, no solo el nuevo caso. Si una prueba deja de pasar, decide si encontraste un defecto o si cambió el contrato y hay que actualizar la expectativa con una explicación. Termina con una nota breve: qué cambió, qué comprobaste y qué límite sigue abierto.

Anota qué parte se puede reutilizar en otro módulo y qué parte depende del contexto de la aplicación. Esa distinción ayuda a decidir si merece la pena extraer una función o componente.

Lección 9 · Repaso activo

Cierra el material y responde sin copiar frases. Si dudas, vuelve a la página indicada, comprueba la fuente primaria y reescribe la respuesta con tus palabras. La recuperación activa ayuda a detectar qué parte aún depende de memorizar un ejemplo concreto.

Preguntas

- ¿Qué problema aborda módulos y paquetes y qué dato necesita?
- ¿Qué diferencia hay entre una solución que funciona en el ejemplo y una que cumple el contrato?
- ¿Qué error frecuente te parece más probable y cómo lo reproducirías?
- ¿Qué alternativa elegirías en un proyecto pequeño y por qué?

Respuestas de comprobación

- La idea clave se resume en esta explicación: Un módulo es normalmente un archivo .py que contiene definiciones importables. import enlaza el módulo o nombres elegidos al espacio actual. Cuando Python ejecuta un archivo directamente, su variable __name__ vale __main__; al importarlo, __name__ toma el nombre del módulo. El bloque de guarda permite ofrecer una acción de demostración sin ejecutarla en cada importación. Un paquete agrupa módulos y su organización debe mantener nombres claros.
- Una fuente inicial de fallo descrita en el tema es: Llamar al archivo como un módulo de la biblioteca estándar que intenta importar.
- Una alternativa es válida si mantiene las entradas, salidas y reglas acordadas y supera las mismas pruebas.

Si tu respuesta no menciona un caso o una condición concreta, añade un ejemplo propio y vuelve a comprobarlo.

Lección 10 · Síntesis y fuentes

Resumen del tema

Has estudiado módulos y paquetes desde el modelo conceptual hasta una práctica comprobable. La secuencia de trabajo es: identificar entradas y contrato, elegir la operación adecuada, seguir el flujo, comprobar casos límite y conservar una prueba para la siguiente modificación. Aplica las reglas de Python sin confundir una característica del lenguaje con una API concreta del entorno.

Lista de control antes de avanzar

- Puedo explicar el concepto con un ejemplo propio y predecir su salida.
- Puedo identificar al menos un caso límite y un error habitual.
- Puedo comparar dos enfoques y justificar mi elección.
- Puedo verificar una solución sin depender de mirar el resultado una sola vez.

Fuentes oficiales para ampliar

Python Software Foundation — Modules

Consulta la sección de la fuente que corresponda a la versión de tu proyecto. Este cuaderno resume conceptos para estudiar; cuando haya una diferencia de detalle, prevalece la documentación oficial del lenguaje o biblioteca.